String Insertion

Several other methods change the text in strings, either in place or returning a new string. The insert method takes an index and another string and inserts the new string at that index. This both mutates the original string and mutates the original string:

```
x = "the pickaxe book"
x.insert(4, "new ") # => "the new pickaxe book"
x                   # => "the new pickaxe book"
```

String Deletion

If you just want to delete a known string from within a larger string, use the delete method. The base method takes one or more character selectors, as shown in Finding Information about a String, on page 518, and returns a copy of the string with matching characters removed. The method delete! does the same thing, but mutates the receiving string and returns the the mutated string. Like other mutation string methods, it returns nil if the string is unchanged:

```
"The pickaxe book".delete("k")   # => "The picaxe boo"
```

You are also able to delete the prefixes and suffixes in strings with delete_prefix and delete_suffix. These methods take actual substrings (not selectors) and remove them from the requested end of the string if that end of the string matches. They return a copy of the string with the change. For example:

```
"The pickaxe book".delete_prefix("The")       # => " pickaxe book"
"The pickaxe book".delete_prefix("Banana")    # => "The pickaxe book"
"The pickaxe book".delete_suffix("book")      # => "The pickaxe "
"The pickaxe book".delete_suffix("podcast")   # => "The pickaxe book"
```

Mutator versions of these methods, delete_suffix! and delete_prefix! return the mutated original string or nil.

String Replacement

It's quite common to want to change a string by changing a specific character or pattern to a different character or pattern. Ruby has a couple of different methods for this.

One family of methods is sub, sub!, gsub, and gsub!. gsub is the most general. The gsub method takes two arguments: a pattern and a replacement. The pattern is a String or a regular expression; the replacement is a String, a Hash, or a block.

The gsub method returns a copy of the receiving string with *all* occurrences of pattern replaced using the replacement value.

The pattern will typically be a Regexp; if it is a String, then no regular expression metacharacters will be interpreted (that is, /\d/ will match a digit, but "\d" will match a backslash followed by a d).

How the replacement value is created depends on the other arguments. If the replacement is a string, the string is put in place of the pattern:

```
"The pickaxe book".gsub("e", "!")       # => "Th! pickax! book"
"The pickaxe book".gsub(/[aeiou]/, "y") # => "Thy pyckyxy byyk"
```

If a string is used as the replacement, special variables from the match (such as $& and $1) cannot be substituted into it, because substitution into the string occurs before the pattern